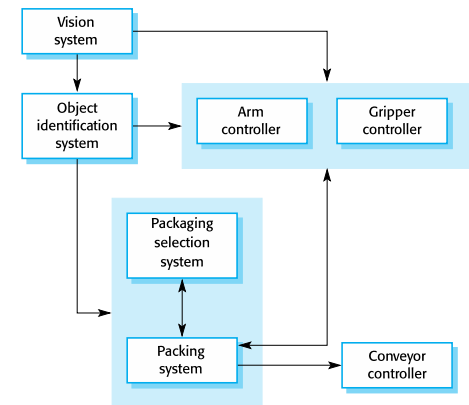
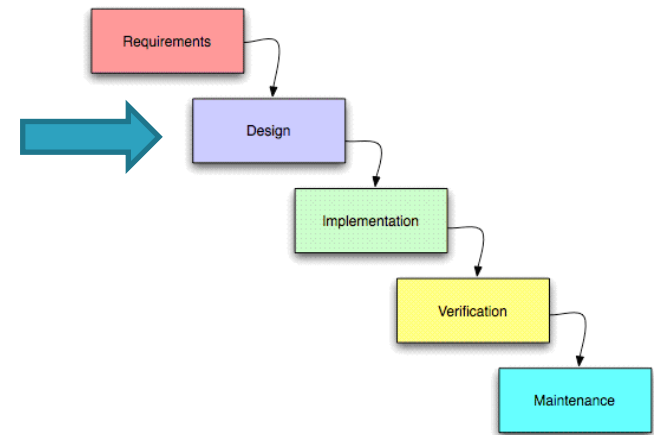


DESIGN ARCHITETTURALE

Ingegneria del Software a.a. 2025-26

AGENDA

- Software Design:
 - Cosa è?
 - Processo creativo?
 - **High level design** vs. **Low level design**
- Design architetturale: Architettura SW
- Concetto di **stile architetturale**
- Alcuni stili architettureali
 - Pipe and Filters
 - Layered
 - Broadcast model
 - ...



Warning 1: Non possiamo vederli tutti, serve vostro approfondimento

Warning 2: Non è possibile andare troppo nel dettaglio ...

DESIGN



- È il processo che:

trasforma un problema in una soluzione!

- **Rispetto ai requisiti:**

- i requisiti definiscono il **problema (cosa)**
- il design propone una **soluzione (come)**

Es. nella costruzione di una casa:

requisiti = richieste dei proprietari-utenti

design = progetto dell'architetto

(schemi e documenti a diverso livello di dettaglio)

- **Rispetto all'implementazione/codifica:**

- il design definisce la **struttura della soluzione**
invece l'implementazione la realizza, rendendola usabile

Es. nella costruzione di una casa:

implementazione = esecuzione dei lavori

(il che include ad esempio la scelta dei materiali usati, ecc.)

LIVELLI DI DESIGN



- Anche il design si sviluppa per raffinamento ...

- **Architectural design**

mappa i requisiti su architettura SW e componenti/sottosistemi

anche system design o high-level design

- **Component design**

fissa dettagli dei componenti, specificando maggiormente la soluzione

anche subsystem design o low-level design

- Scelte tecnologiche

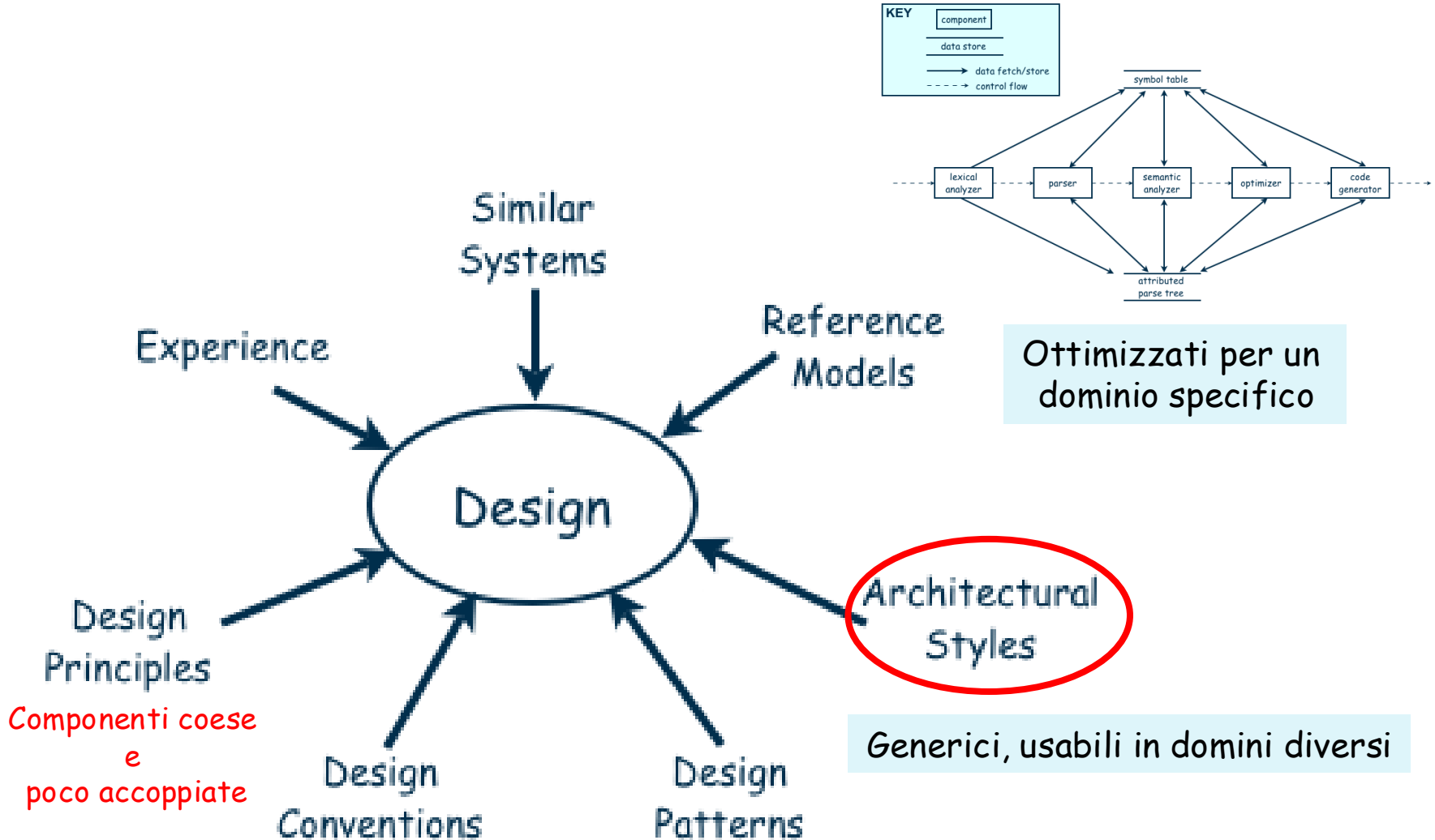
- **platform-independent design**

- **platform-specific design (es., .Net o J2EE)**

DESIGN: UN PROCESSO CREATIVO?

- Il design può essere migliorato studiando **esempi** di buon design
- La maggior parte del lavoro di design è lavoro *di routine*
 - i problemi vengono risolti **riutilizzando ed adattando soluzioni a problemi simili (ri-uso)**
- Tre diversi livelli di **ri-uso**:
 - **Clonazione**: Si riutilizza interamente design/codice, con piccoli aggiustamenti
 - **Design pattern**: Buona soluzione a problema ricorrente
 - Li vediamo prossimamente ...
 - **Stili architettureali**: Architettura generica che suggerisce come decomporre il sistema

DESIGN: UN PROCESSO CREATIVO?



DESIGN ARCHITETTURALE

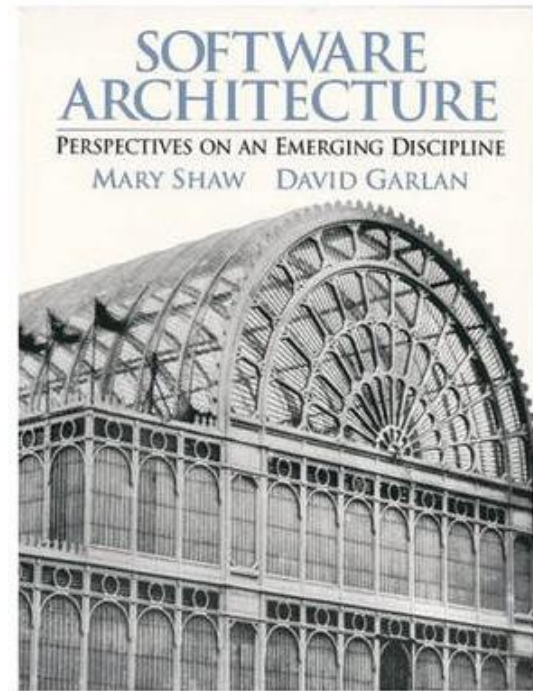
◦ Design architetturale:

processo di design per identificare:

- le (macro) componenti di un sistema
 - Non siamo a livello di classi, strutture dati e algoritmi!
- il framework per il controllo e la comunicazione tra componenti

◦ Produce una descrizione dell'**architettura software**

- per analogia: corrisponde al progetto architettuale di una casa
- concetto introdotto da Mary Shaw e David Garlan nel 1996



VANTAGGI NELL' AVERE UN' ARCHITETTURA SW

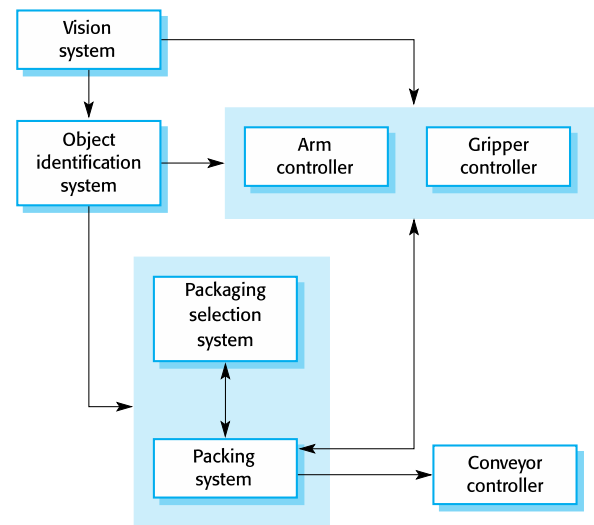
- Guida lo sviluppo ed aiuta nella comprensione del sistema
 - la realizzazione di più componenti distinti è meno complessa della realizzazione di un sistema come monolito
- Supporta decisioni manageriali
 - ad esempio se rifattorizzare il sistema o no
- Facilita l'analisi di alcune proprietà
 - alcune proprietà sono soddisfatte?
 - Es. manutenibilità, efficienza
- Permette il riuso (Large-scale)
 - l'architettura stessa può essere riutilizzabile in più sistemi
- Aiuta a ragionare sull'evoluzione del sistema
- Documenta il sistema
 - Serve a capire le dipendenze tra componenti
 - Elemento fondamentale della documentazione del sistema

DIAGRAMMA A BLOCCHI

- Un architettura software è normalmente espressa mediante un **diagramma a blocchi** che presenta un “overview” della struttura del sistema
- I **blocchi** sono i componenti
 - Sottosistemi o moduli
- I **connettori** rappresentano le “relazioni” tra i componenti
 - Es. A e B comunicano scambiandosi dei messaggi

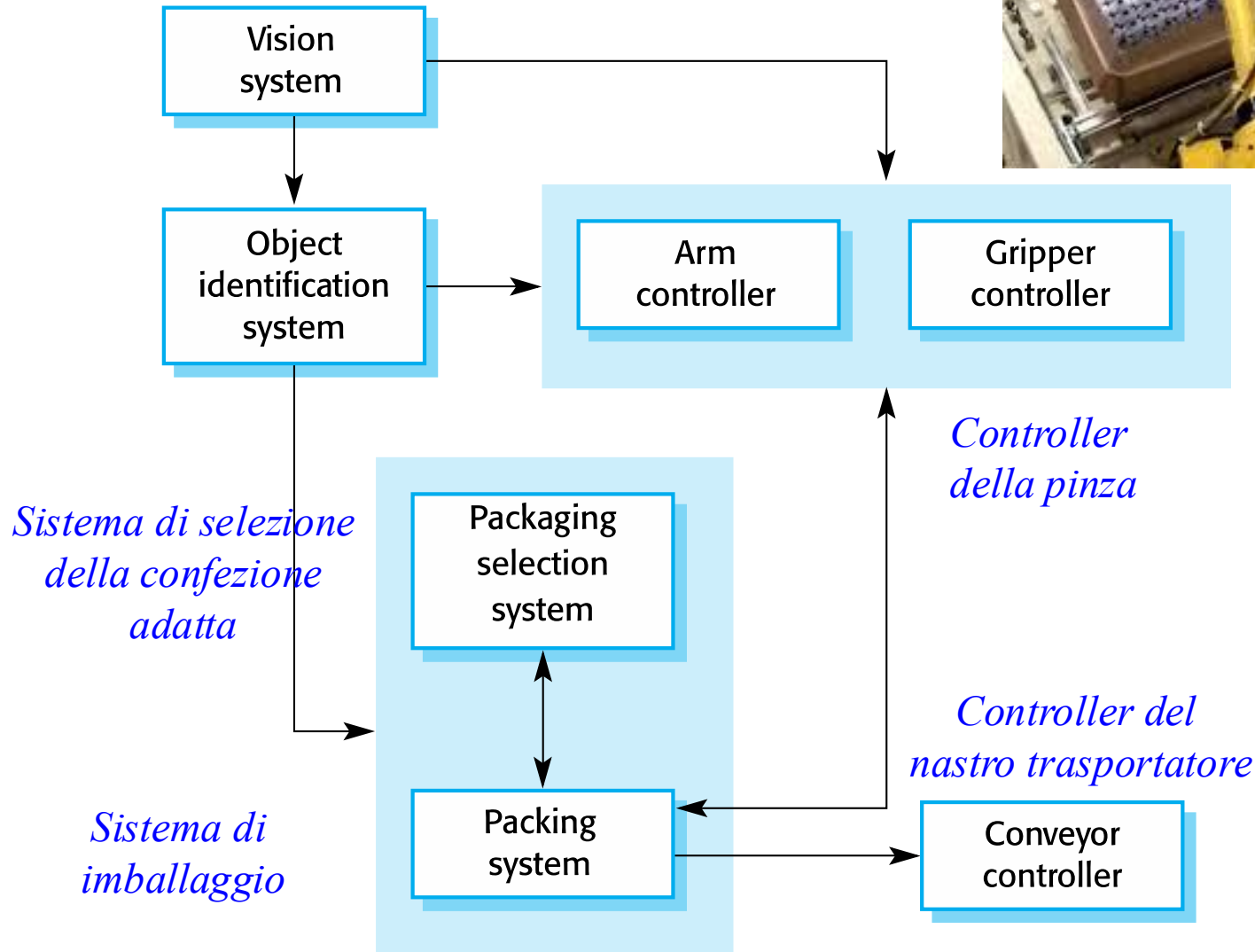
Esistono anche linguaggi formali di specifica architetturale, es. Darwin, ADLs

UML fornisce il diagramma delle componenti!!!



ARCHITETTURA SW: ESEMPIO

Sistema di controllo di imballaggio robotizzato



ARCHITETTURA SW: PROPRIETÀ DEL SISTEMA

Grosso impatto dell'architettura su:

- **Performance** (es. tempo di risposta): Minimizzare le comunicazioni
 - Componenti **large-grain** piuttosto che **fine-grain**
- **Security:** Usare un'architettura a livelli con le risorse critiche nei livelli più interni
- **Safety:** Localizzare le caratteristiche safety-critical in pochi sottosistemi
- **Availability** (24/7): Includere componenti ridondanti e meccanismi per fault tolerance
- **Maintainability:** Usare componenti **fine-grain** facilmente rimpiazzabili

ARCHITETTURA SW: PROPRIETÀ DEL SISTEMA

Grosso impatto dell'architettura su:

- **Performance** (es. tempo di risposta): Minimizzare le comunicazioni
 - Componenti **large-grain** piuttosto che **fine-grain**
- **Security**: Usare un'architettura a livelli con le risorse critiche nei livelli più bassi
- **Safety**: Includere caratteristiche safety-critical nei sistemi
- **Availability** (24/7): Includere componenti ridondanti e meccanismi per fault tolerance
- **Maintainability**: Usare componenti **fine-grain** facilmente rimpiazzabili

Tirare la coperta da una parte per scoprirsi dall'altra

ARCHITETTURA SW: PROPRIETÀ DEL SISTEMA

Grosso impatto dell'architettura

- **Performance** (es. tempo di esecuzione) minimizzare le comunicazioni
 - Componenti **large-grain** a costo che **fine-grain**
- **Security** architettura a livelli con le risorse critiche nei livelli più interni
- **Safety**: Localizzare le caratteristiche safety-

Trade-off:

- Componenti **large-grain** migliorano performance ma riducono maintainability
- Componenti **ridondanti** migliorano availability ma peggiorano safety ...
- Usare un'architettura a livelli **migliora** security ma aumenta comunicazioni e così degrada performance ...

Tirare la coperta da una parte per scoprirsi dall'altra

STILE ARCHITETTURALE

- L'architettura di un sistema può **conformarsi** a uno **stile architettuale**
 - Modello generico con caratteristiche specifiche che può essere istanziato/personalizzato

Lo **chalet alpino** è caratterizzato dall'uso di materiali naturali come legno e pietra, con travi a vista, balconi in legno decorati ...



- **Conoscere gli stili architettureali può semplificare il problema di definire l'architettura software**
 - In realtà, la maggioranza dei grandi sistemi sono **eterogenei** e non seguono un singolo stile architettureale

ELEMENTI DI UNO STILE ARCHITETTURALE

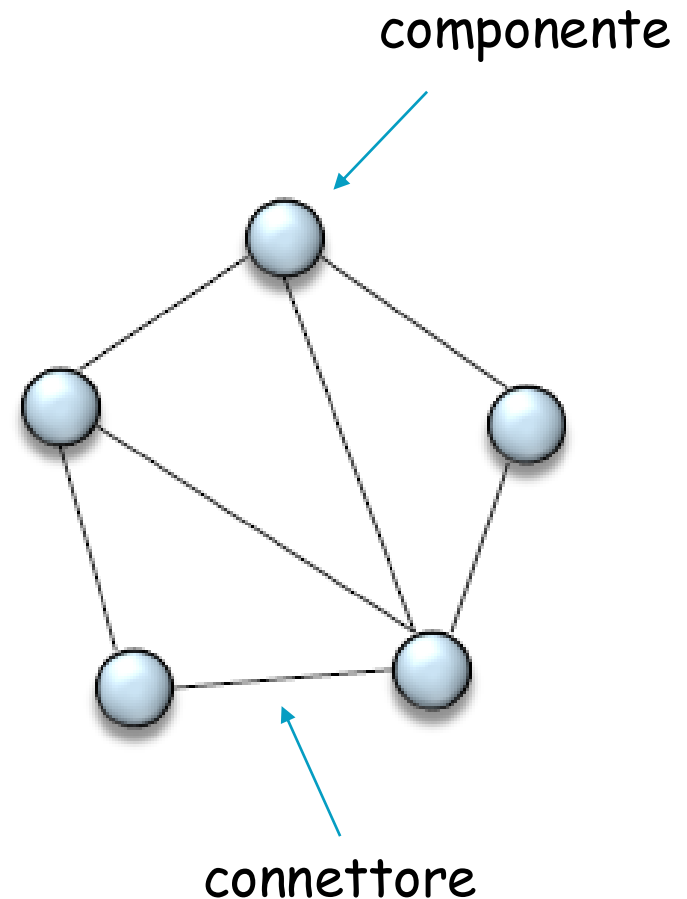
○ Uno stile architetturale è definito da

- **tipo di componenti** architetture di base

- es. procedure, filtri, livelli/strati, servizi (web service), classi, package di classi, sotto-sistemi

- **tipo di connettori**

- es. invocazioni di procedure/metodi, pipe, eventi, scambio di messaggi



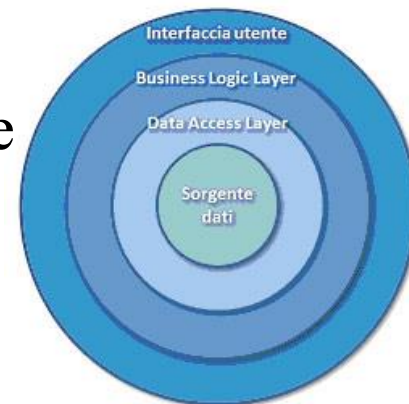
RICAPITOLANDO ...



produce



(può essere)
conforme



Design architetturale
(Fase di high level design)

Architettura SW

Stile architetturale

STILI ARCHITETTURALI

Warning:
non li vedremo tutti!

○ Diversi stili:

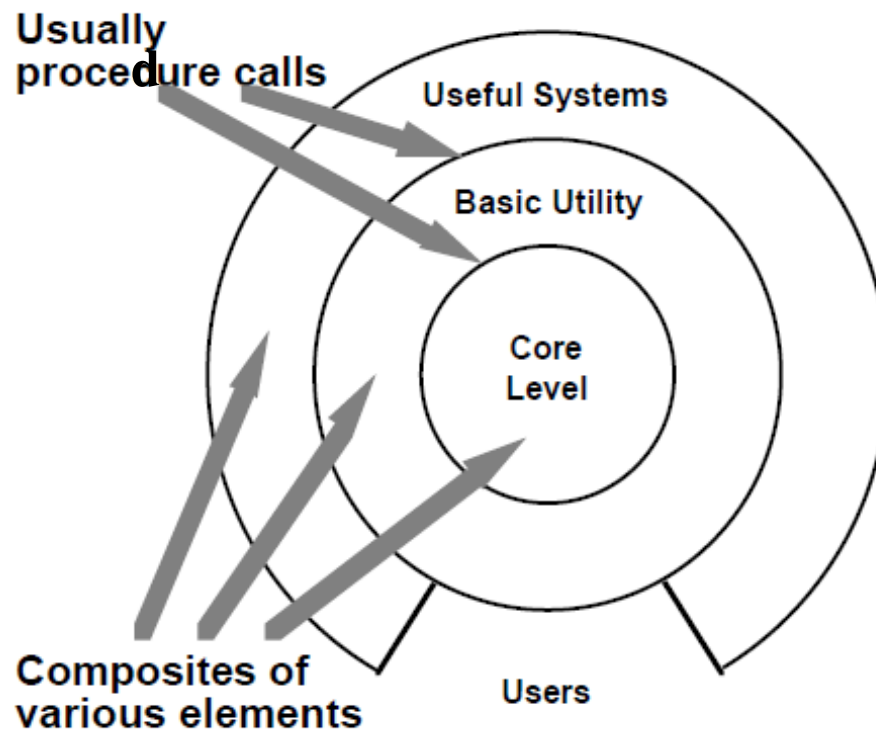
- Layered (Stratificato)
- Repository
- Client/server
 - Two tier / three tier
- P2P
- Pipe and Filter
- Broadcast Model
- Service Oriented Architecture
- Microservice
- ...

○ Stile strutturale vs Stile di controllo

- **Strutturale**: solo info strutturali
- **Di Controllo**: anche info (o solo) di controllo
 - Il modulo 'xyz' controlla 'xx' e 'zz'?

STRATIFICATO (LAYERED)

- Organizza il sistema in un **insieme di livelli** ognuno dei quali fornisce un insieme di servizi
- Un livello “si appoggia” solo sul livello inferiore
 - Cioè usa solo i servizi del livello inferiore



Buon esempio di Layered model!



ESEMPIO: ANDROID



https://www.tutorialspoint.com/android/android_architecture.htm

STRATIFICATO: PRO E CONTRO



○ Vantaggi:

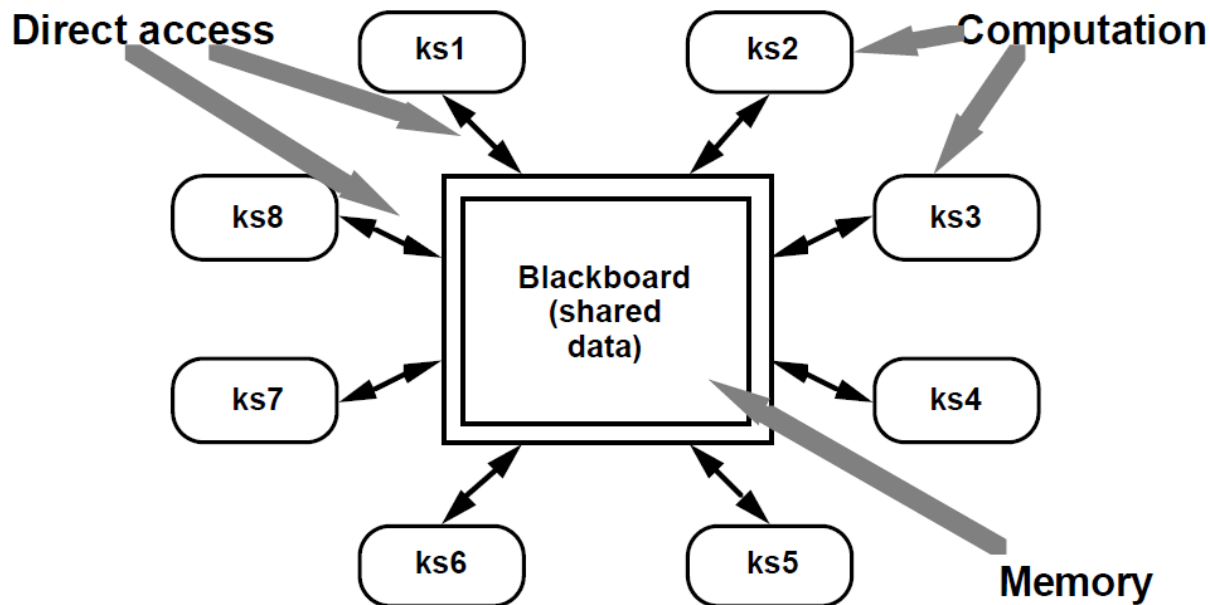
- Se cambia l'interfaccia di un livello, solo il livello adiacente ne è influenzato
- **Supporta il riuso**: differenti implementazioni dello stesso livello possono essere usate in modo intercambiabile

○ Svantaggi:

- Spesso però è artificiale strutturare il sistema in questo modo
- Può risultare inefficiente (**pila di chiamate!**) e spesso questo porta a “shortcut”
 - Si salta qualche livello (e allora si perdono i vantaggi)

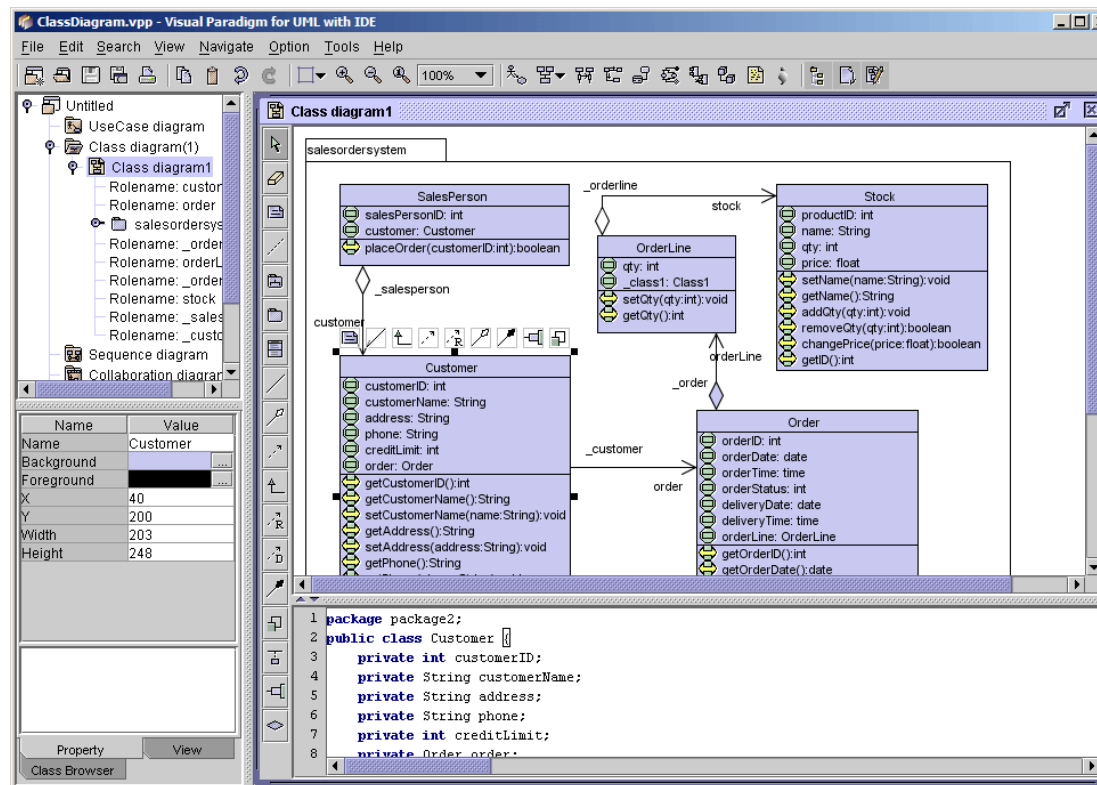
REPOSITORY MODEL

- I **dati condivisi** sono mantenuti in un **database centrale (repository)** a cui hanno accesso tutti i sotto-sistemi
 - No comunicazione diretta tra sotto-sistemi!
- Adatto per applicazioni in cui i dati sono prodotti da un sottosistema e utilizzati da un altro

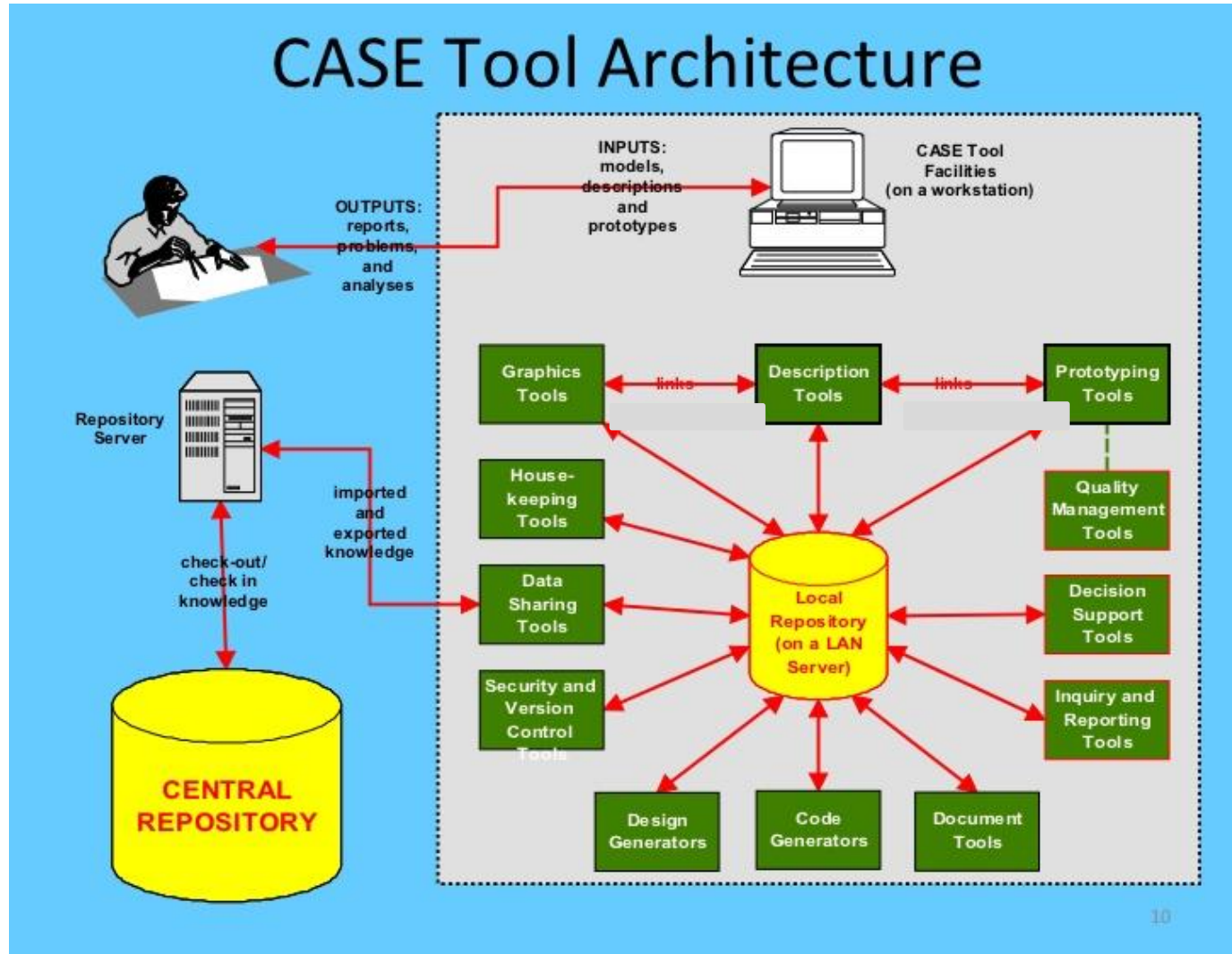


ESEMPIO: CASE TOOL (1)

- I CASE sono tool che supportano lo **sviluppo** e **manutenzione** del software
 - Non solo fase di codifica ma spesso anche design e requisiti



ESEMPIO: CASE TOOL (2)



Integrated database repository for storing all artifacts of the software process

REPOSITORY MODEL: PRO E CONTRO



○ Vantaggi

- Modo efficiente di condividere grandi quantità di dati (non sono trasmessi esplicitamente tra sottosistemi)
- Gestione centralizzata di backup, security, etc.

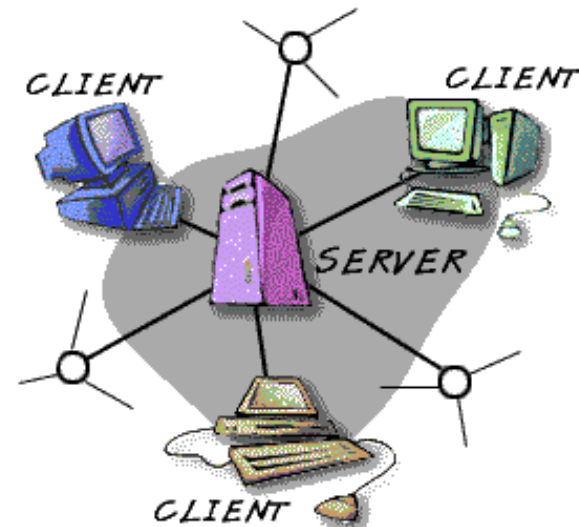


○ Svantaggi

- I sottosistemi devono accordarsi su un modello dei dati per il repository, necessità di compromesso
 - Modello dei dati (schema) è unico
- L'evoluzione dello schema dei dati è difficile e costosa
- Difficile da rendere distribuito in maniera efficiente

CLIENT-SERVER MODEL

- Modello di **sistema distribuito** che mostra come i **dati** e la **computazione** possono essere distribuiti su:
 - insieme di **server** che **forniscono servizi** specifici
 - Es. servizi di stampa, servizi di gestione file,
 - insieme di **client che utilizzano tali servizi**
- Esiste una **rete** che permette ai client di accedere ai server



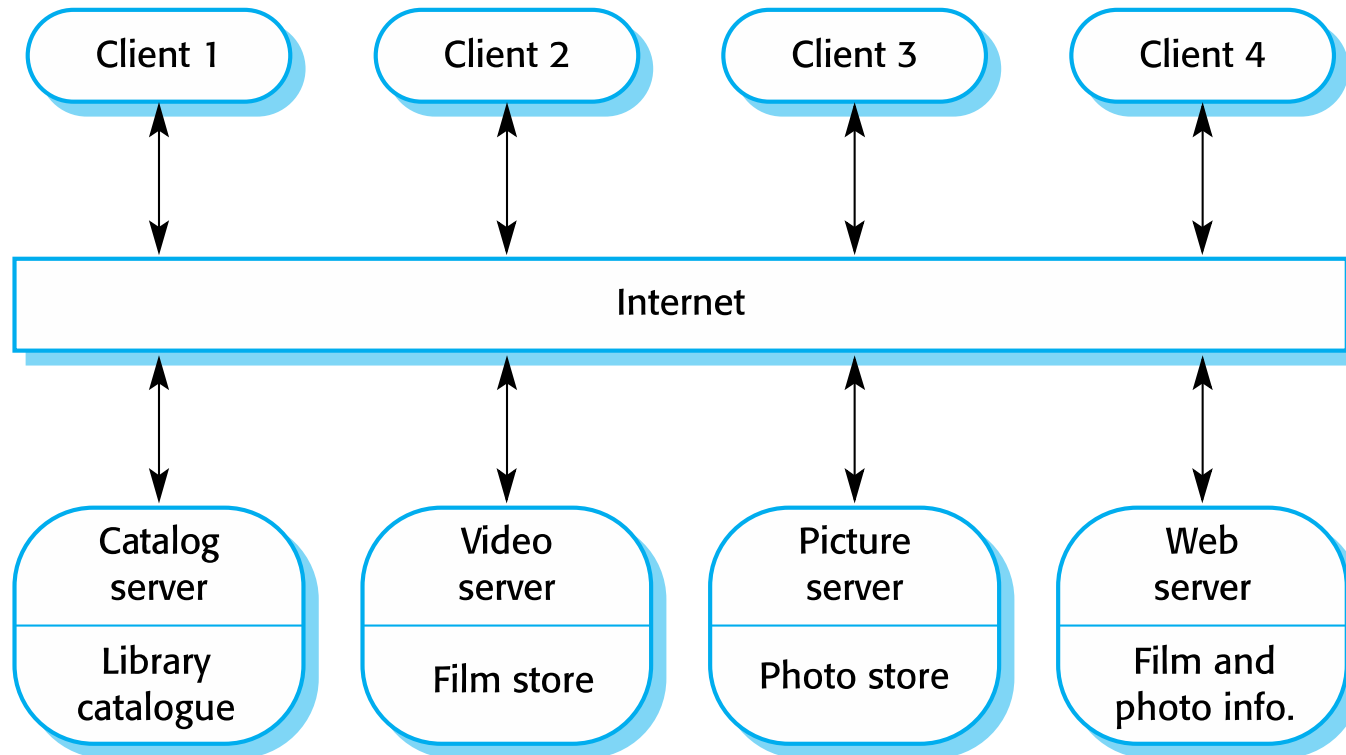
Protocollo 'request and response'

ESEMPIO: WEB APP PER FILM E FOTOGRAFIE



Interfaccia utente costruita usando **Web browser**

MovieStillsDB



Il server catalogo gestisce il catalogo e vendita prodotti

Il server video gestisce compressione e decompressione (bassa risoluzione)

Le immagini sono ad alta risoluzione per cui ha senso gestirle su un altro server

CLIENT-SERVER MODEL



○ Vantaggi

- Possibile migliorare le performance di un sistema “spostando” la computazione tra client e server e viceversa
- **Supporta il riuso**: server che forniscono un servizio possono essere riutati in diverse applicazioni
- **Scalabile**: si aggiungono dei server
- Aggiunta di nuovi server o upgrade di quelli esistenti è semplice

○ Svantaggi (è un sistema distribuito!)

- Mancanza di modello dei dati condiviso tra server, lo scambio di dati può essere inefficiente
- Operazioni di gestione ridondanti nei vari server
- Problema di gestione: i server potrebbero essere di proprietà di differenti organizzazioni.

CLIENT SERVER A DUE LIVELLI (TWO-TIER)

Principio di buon design

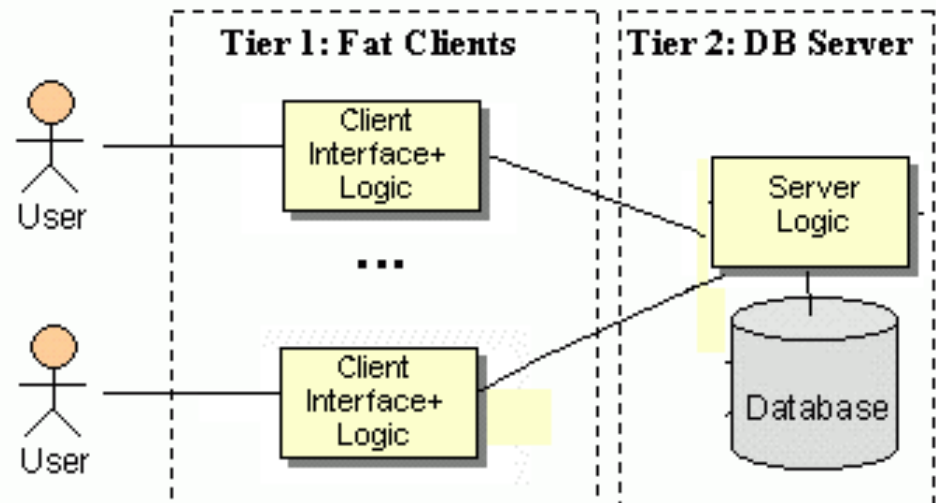
Tre componenti software:

- **Interfaccia utente (presentation logic)**
- **Gestione dei processi e logica (business logic)**
- **Gestione del DB (data logic)**

distribuiti in due strati:

- **Client** (chi richiede il servizio)
- **Server** (chi fornisce il servizio)

Thin-client vs Fat-client



CLIENT-SERVER A TRE LIVELLI (THREE-TIER)

Evoluzione del Two-Tier

- sempre client-server
- sul client resta solo l'interfaccia utente
- la logica del sistema è uno strato separato che gestisce **multi-utenti**
- gli strati di logica e gestione DB sono distribuiti su più server

Client

Servers

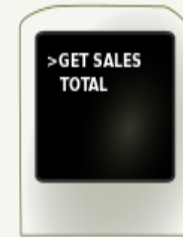
Servers

Application server e DB server

Quante vendite totali?

Presentation tier

The top-most level of the application is the user interface. The main function of the interface is to translate tasks and results to something the user can understand.



Logic tier

This layer coordinates the application, processes commands, makes logical decisions and evaluations, and performs calculations. It also moves and processes data between the two surrounding layers.



GET LIST OF ALL SALES MADE LAST YEAR



ADD ALL SALES TOGETHER

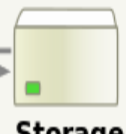


Data tier

Here information is stored and retrieved from a database or file system. The information is then passed back to the logic tier for processing, and then eventually back to the user.



Database

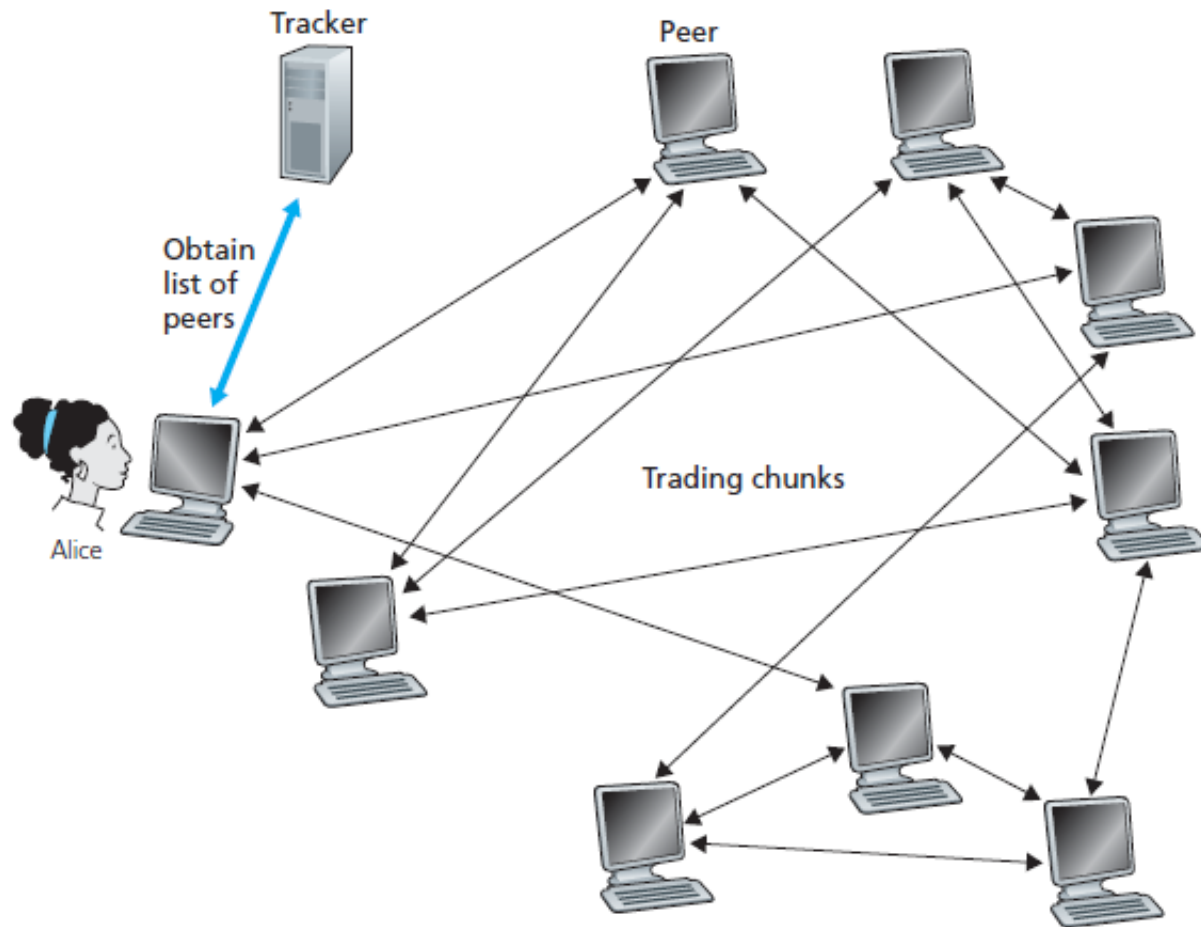


Storage

PEER-TO-PEER (P2P) MODEL

- Ogni componente esegue il suo processo e gioca allo **stesso tempo** il **ruolo di client e server**
- Ogni peer ha un interfaccia che specifica i servizi offerti (server) e quelli richiesti (client)
- Peer comunicano richiedendo servizi ed offrendoli
 - Simile a client-server, ma qui i ruoli non sono definiti
- **Cosa distingue un peer da un altro sono i dati!**
 - Se un peer ha bisogno di un dato lo chiede ad un altro peer
- I sistemi P2P **scalano bene** e sono **fault-tolerant**
 - Aggiungere un “peer” vuol dire aggiungere nuove capabilities (in termini di nuovi dati)
 - I dati sono replicati su diversi peer per cui se un nodo viene sconnesso per una qualche ragione non si perde (molta) informazione

ESEMPIO: bitTorrent



ESEMPIO: eMule

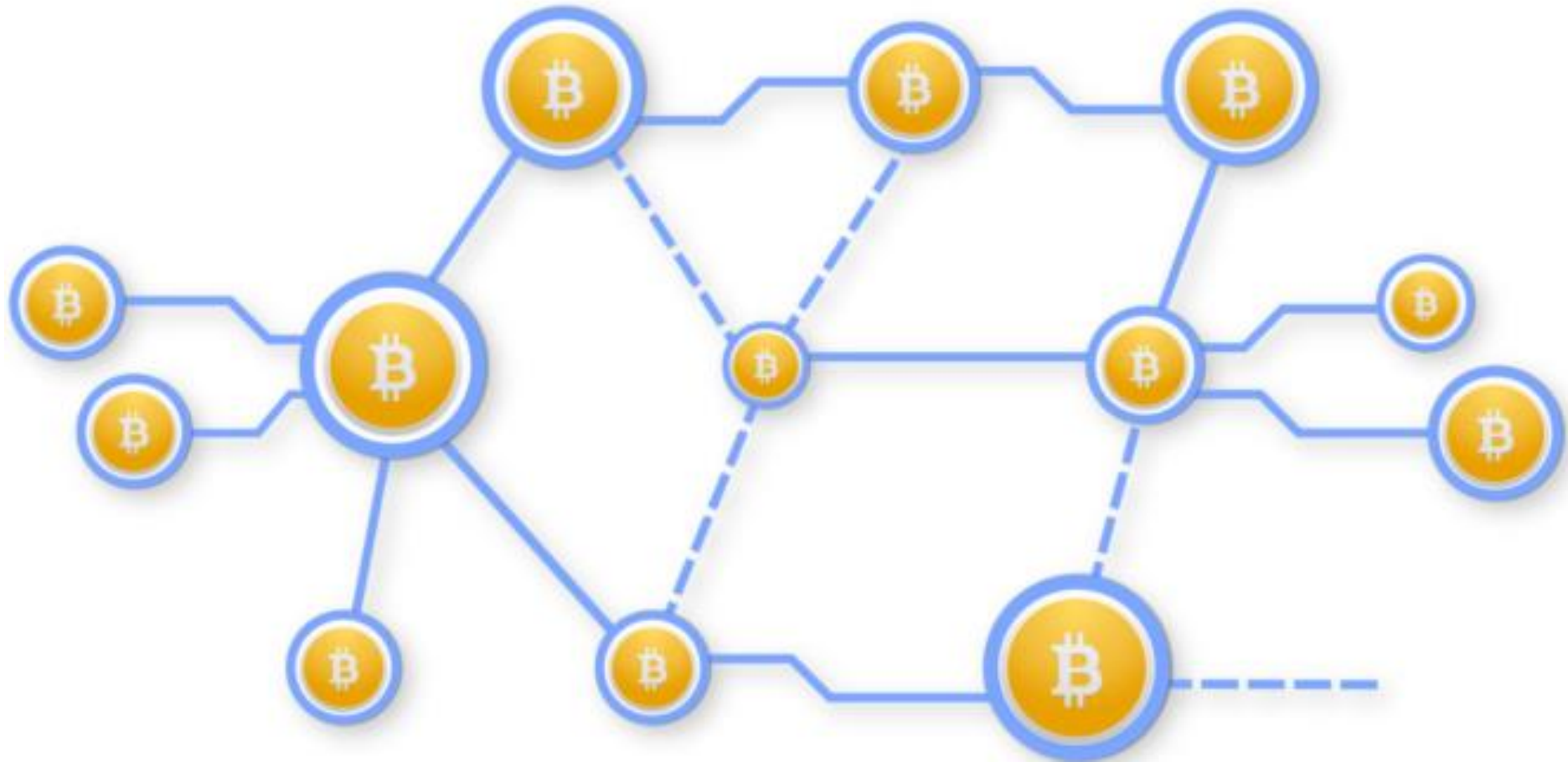


Altri: BitTorrent e Vuze



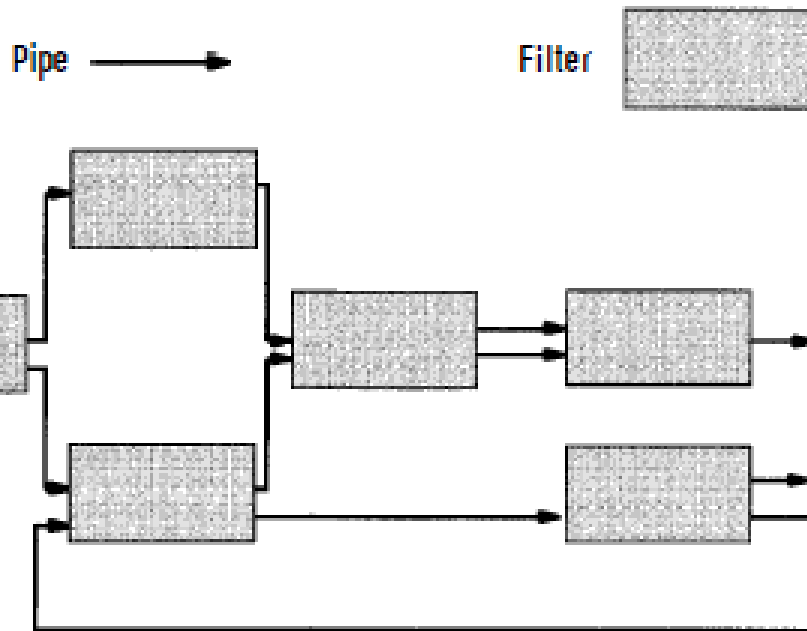
eMule è un software applicativo open source dedicato alla **condivisione file** appoggiandosi su una rete peer to peer e scritto in linguaggio di programmazione C++ per il sistema operativo Microsoft Windows ...

ESEMPIO: BITCOIN



PIPE AND FILTER

- I **filtri** effettuano **trasformazioni** che elaborano i loro input per produrre output
- Le **pipe** sono **connettori** che trasmettono i dati tra filtro e filtro



Esempi ben conosciuti di pipe and filter sono i programmi scritti nella **shell UNIX**

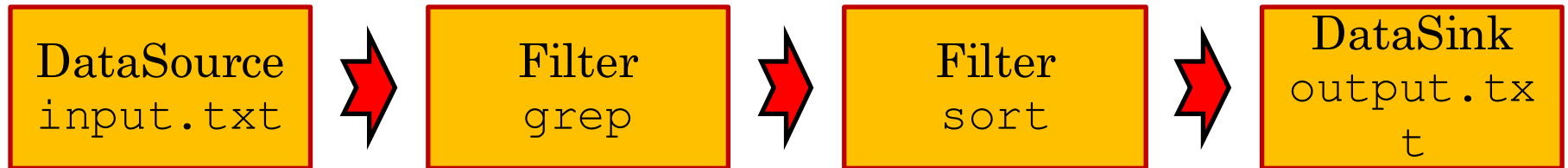
```
ls -l | grep "Aug"
```

```
-rw-rw-rw- 1 john 11 Aug 6 14:10 ch02  
-rw-rw-rw- 1 john 85 Aug 6 15:30 ch07  
-rw-rw-r-- 1 john 12 Aug 15 10:51 intro
```

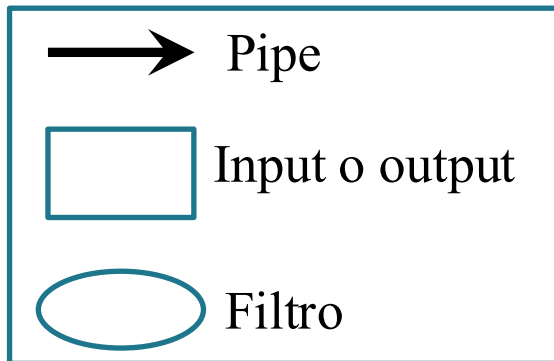
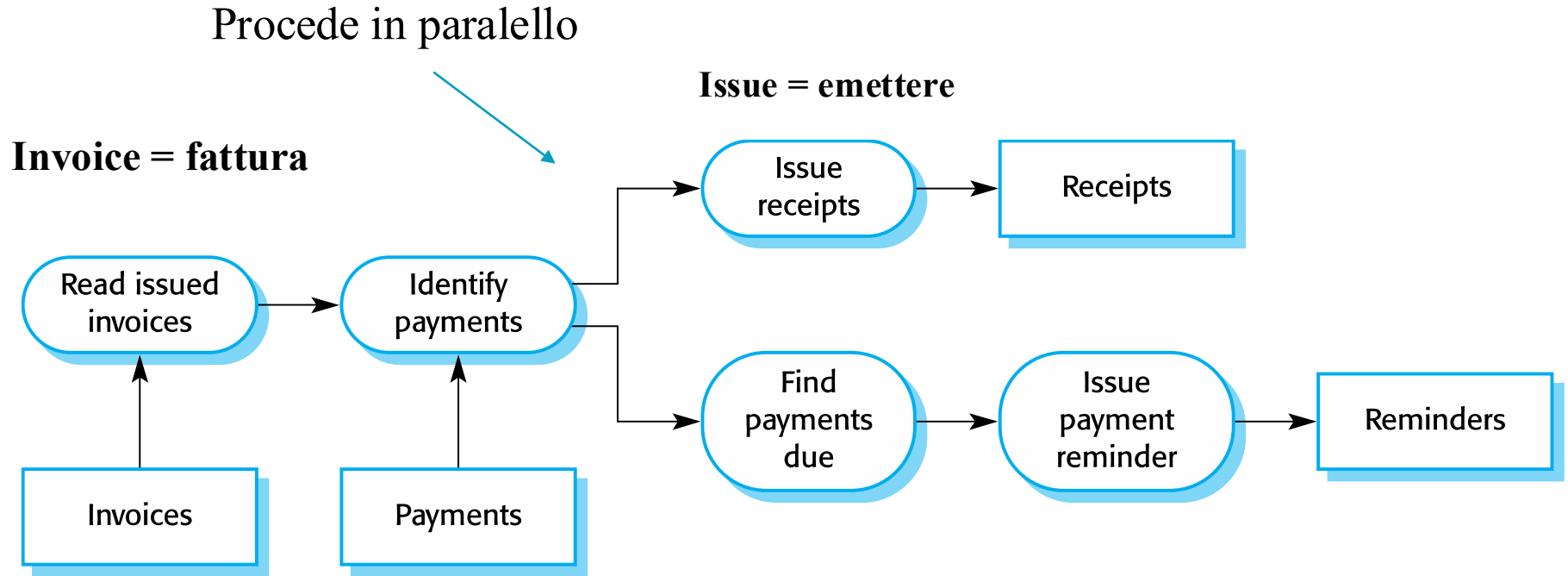
ESEMPIO: PIPE UNIX

```
Cat input.txt | grep "Italy" | sort > output.txt
```

Pipes

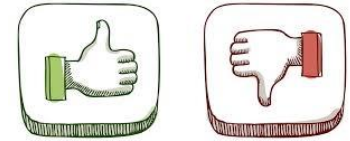


ESEMPIO: INVOICE PROCESSING SYSTEM



PIPE AND FILTERS MODEL: PRO E CONTRO

○ Vantaggi



- Supporta il riuso delle trasformazioni
- **Intuitivo**: permette di capire il funzionamento del sistema come composizione dei filtri
- Aggiunta di nuove trasformazioni facile
- Abbastanza semplice da implementare
 - anche con linguaggi OO
- Supporta esecuzione concorrente/parallela

○ Svantaggi

- Non adatto per sistemi interattivi
 - Click del mouse e selezioni dei menu?
- Di solito porta allo sviluppo di **sistemi batch** dove ogni filtro compie una completa trasformazione dell'input
- Lavoro extra per **parsing** e **unparsing** dei dati nei vari filtri (può non essere efficiente)

STILI DI CONTROLLO

- Esistono due stili di controllo:
 - **Controllo centralizzato:** un sottosistema ha la responsabilità generale del controllo e “avvia e ferma” gli altri sottosistemi
 - **Call – return model**
 - **Manager model**
 - **Controllo basato su eventi:** non esiste la figura di “controllore”; ogni sottosistema può rispondere ad eventi generati da altri sottosistemi
 - **Broadcast model** (es. MQTT protocol)



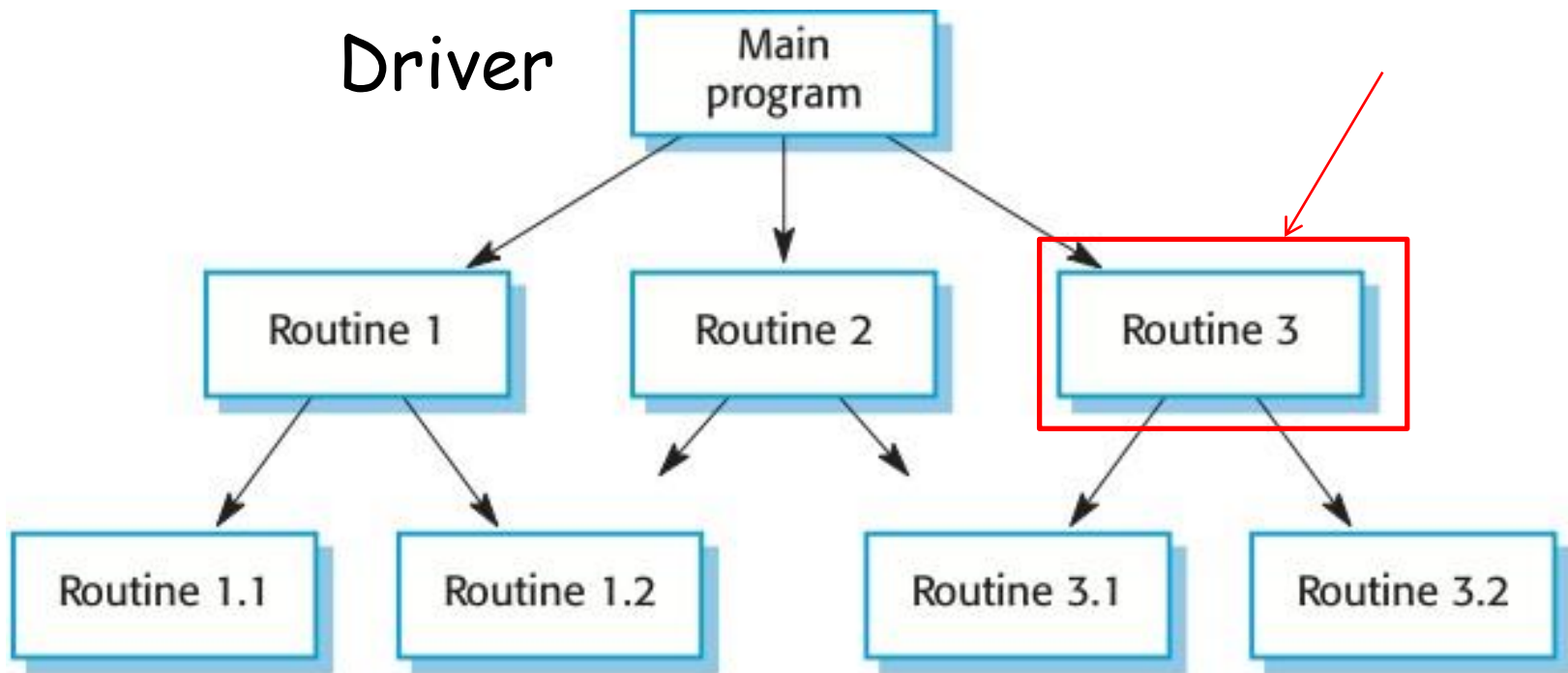
CALL-RETURN MODEL



- Un sottosistema di controllo (**Main**) ha la responsabilità di gestire l'esecuzione degli altri sottosistemi
 - Anche chiamato “**Main Program – Subroutine**”
- Il Main decide l'ordine di esecuzione dei sottosistemi
 - E' il **Driver** (i sottosistemi sono le routine)
- Modello top-down in cui **il controllo** parte dalla radice e si sposta verso il basso
- **Applicabile solo a sistemi sequenziali**

CALL-RETURN MODEL

Solo un sottosistema (quello che ha il controllo) è in esecuzione



MANAGER MODEL

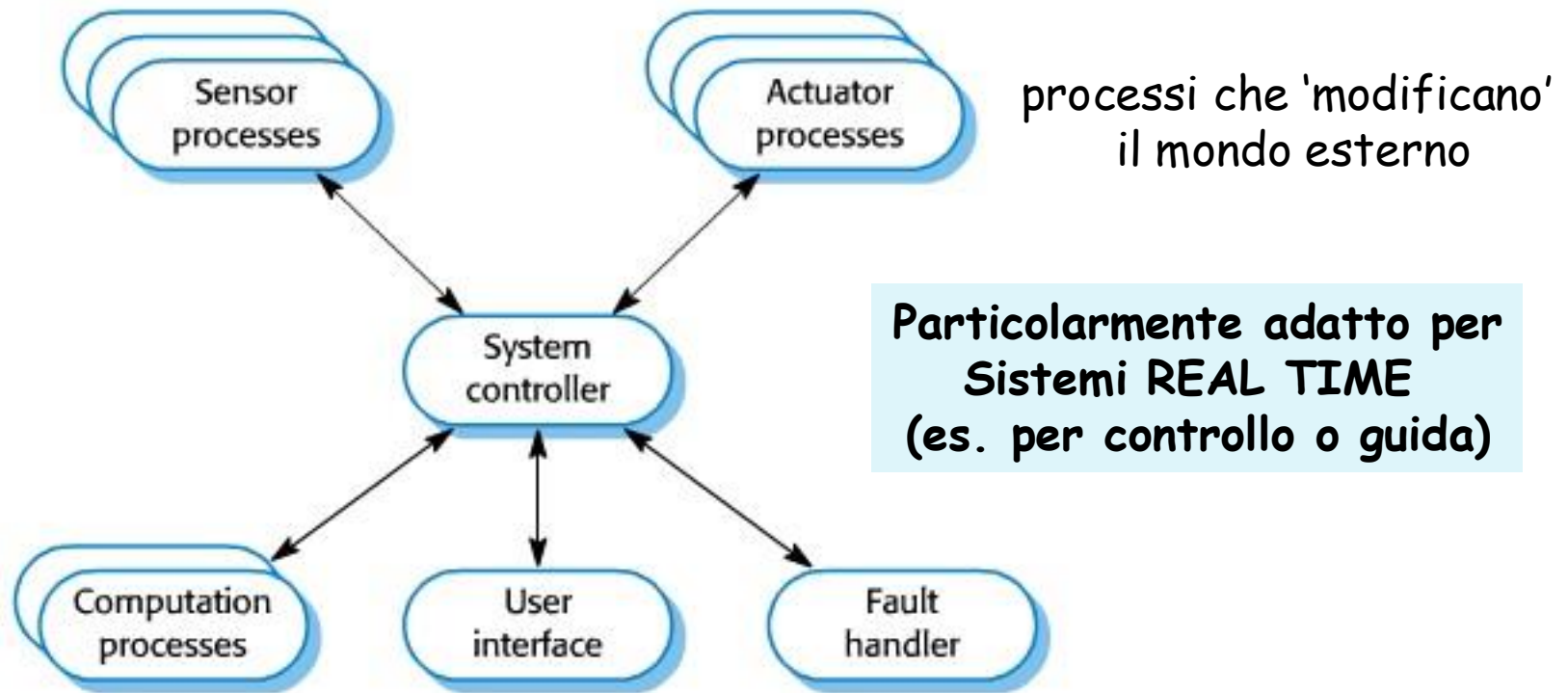
- Se i sottosistemi controllati possono funzionare in **parallelo** allora è possibile usare lo stile Manager model
- Una componente del sistema (**Il Manager**) controlla l'avvio, la terminazione e il coordinamento degli altri processi di sistema
- **Applicabile a sistemi concorrenti**



“Un attuatore è un apparecchio che traduce un segnale elettrico in movimento meccanico”

MANAGER MODEL

Il manager (system controller) itera continuamente interrogando i Sensori, UI e output dei processi per identificare eventi o cambi di stato e manda in esecuzione i processi attuatori e di computazione...



ARCHITETTURA SW: SISTEMA ALLARME BANCA

Sensori di movimento nelle stanze



Processo dei sensori di movimento

Sensori alle porte



Processo dei sensori alle porte

Sensori alle finestre



Processo dei sensori Alle finestre

N° stanza

N° porta

N° Finestra

Manager/Controller

Processo di monitoraggio della banca

N°, tipo, posizione

N°, tipo, posizione



addetto

Processo del sistema di allarme

Processo di comunicazione

Messaggio vocale

Processo dell'allarme acustico



altoparlante

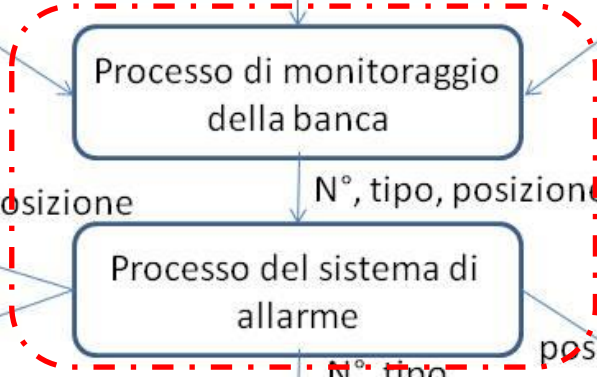
Processo controllo luci



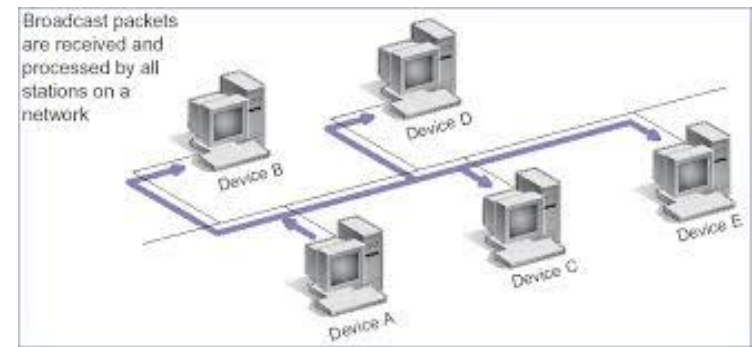
luci

Processo del sintetizzatore vocale

Attuatori

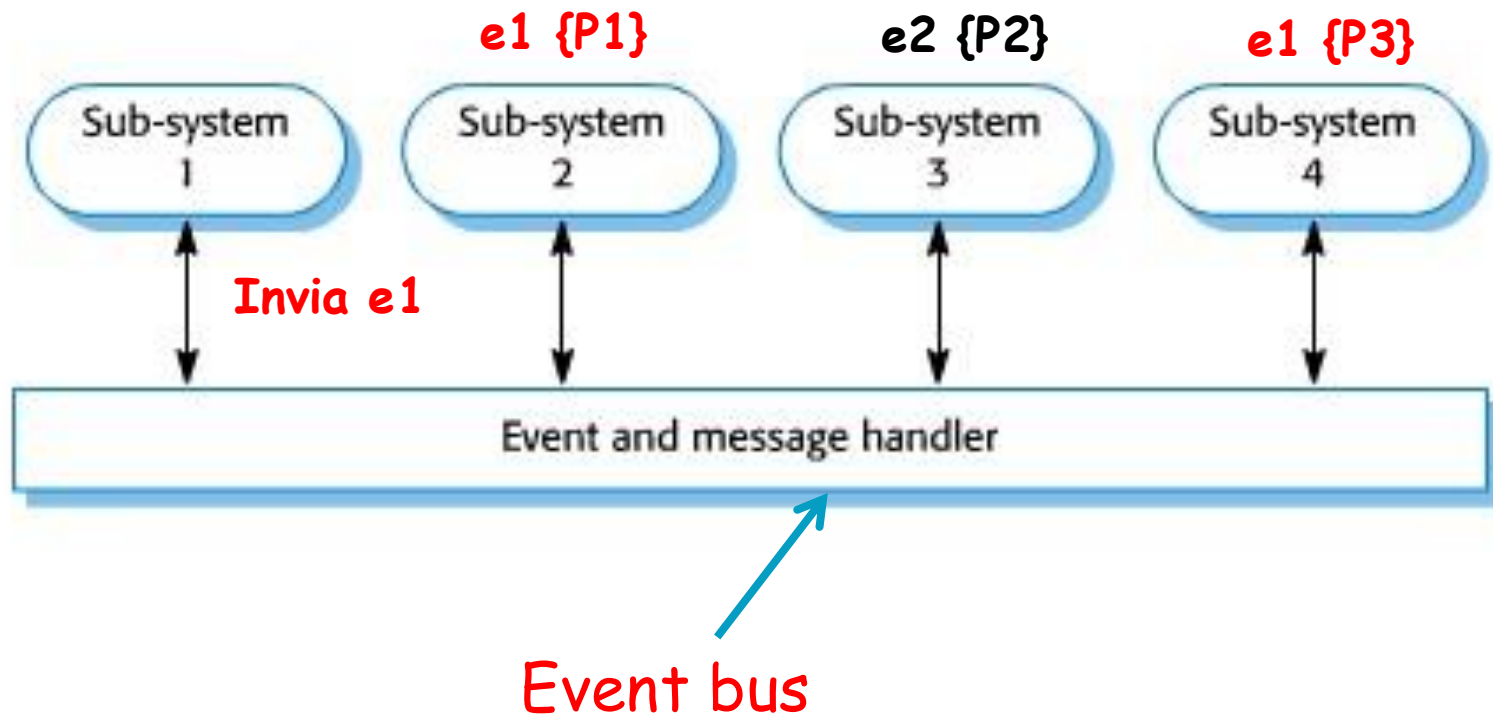


BROADCAST MODEL



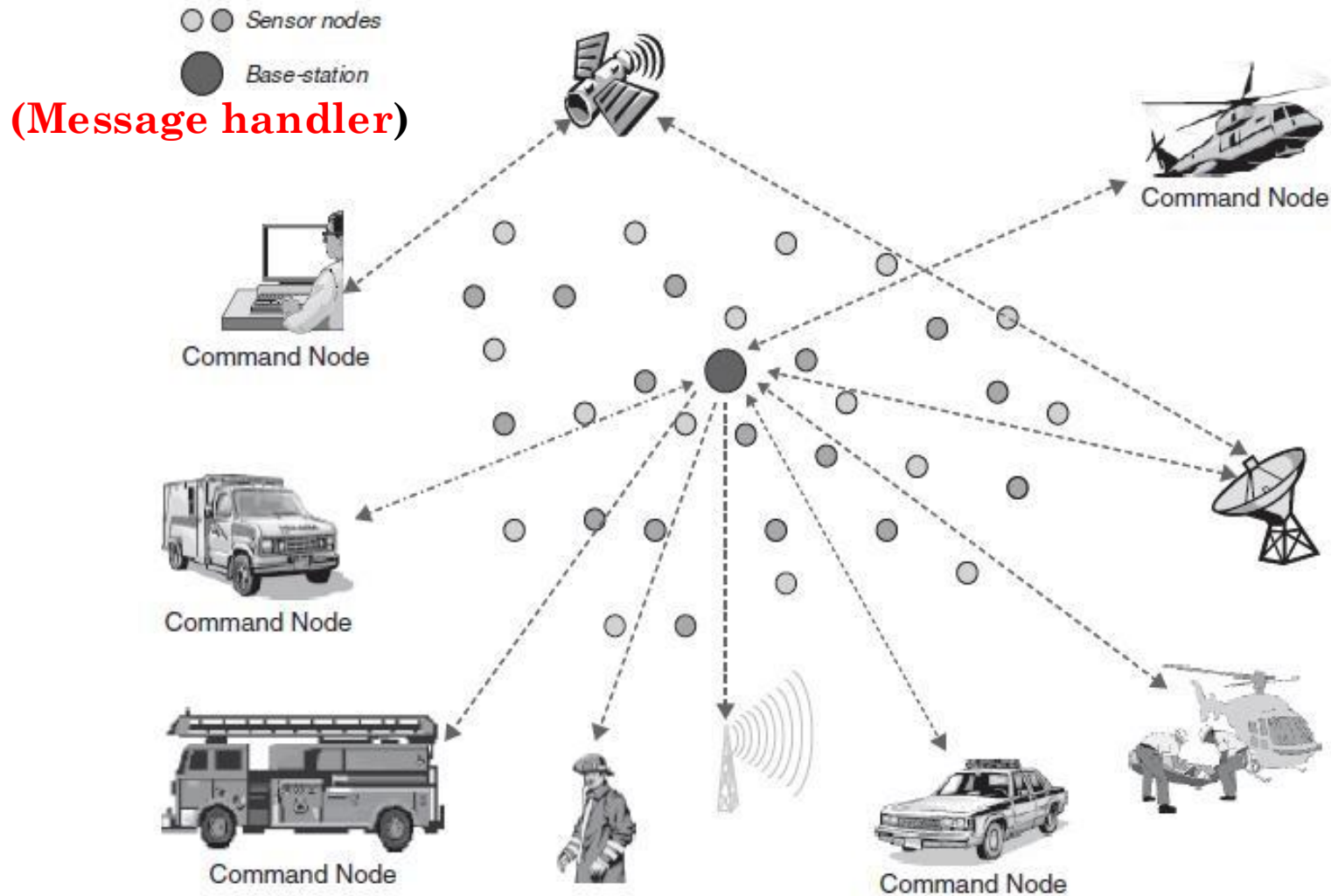
- Invece di invocare una procedura direttamente, un componente può annunciare (**broadcast**) uno o più eventi
 - Un evento è inviato a tutti i componenti
 - Ogni componente in grado di gestirlo può farlo
- **Tutti i componenti sono in ascolto**, quelli interessati all'evento eseguono la procedura associata all'evento
- Così implicitamente un evento causa invocazioni di procedure in altri moduli
- Anche conosciuto come Event based (implicit invocation) o **Publish-Subscribe**

BROADCAST MODEL



Quando Sub1 invia $e1$ le procedure $P1$ e $P3$ sono eseguite

SISTEMI MILITARI E SISTEMI DI SORVEGLIANZA



BROADCAST MODEL: PRO E CONTRO



○ Vantaggi

- Semplice aggiungere/togliere/sostituire componenti
- Semplice il riuso di componenti
 - Un componente può essere riusata in diversi sistemi

○ Svantaggi

- I sottosistemi quando mandano un evento non sanno se e quando l'evento verrà gestito
- Scambio dei dati può complicare le cose
 - I dati sono passati con l'evento
 - Utilizzo di un repository
 - Problemi di performance
- Possono essere difficili da implementare e **soprattutto testare**

ARCHITETTURE ETEROGENEE

- Fino adesso abbiamo parlato solo di **stili architetturali “puri”** ...
- Ci sono due modi di **combinare** gli stili ottenendo così un **architettura eterogenea**
 - **Modo gerarchico**
 - Es. Un sistema organizzato a pipe and filter, di cui una componente interna è a sua volta organizzata come sistema a livelli (layered)
 - Permettendo che una **componente** sia un **mix di architetture**
 - Es. Una componente che accede ad un repository ma allo stesso tempo interagisce attraverso pipe con un'altra

Vestigial data flow



Data fetch/store

Mix di architetture: unisce "pipe and filters" e repository

MICROSERVICES

Veloce panoramica dello stile architetturale

COSA SONO?



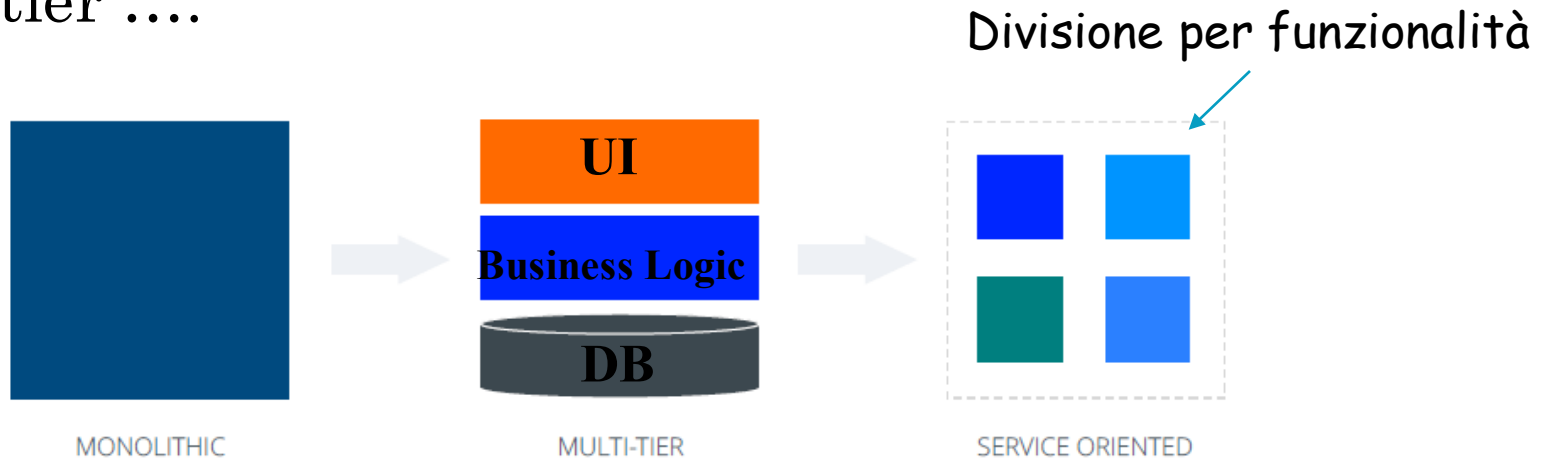
In short, the microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API.

James Lewis – Martin Fowler, "Microservices"

Evoluzione di architetture **SOA**
(service oriented programming)

PERCHÉ SONO NATI?

- In contrapposizione alle applicazioni 'monolite' e multi-tier

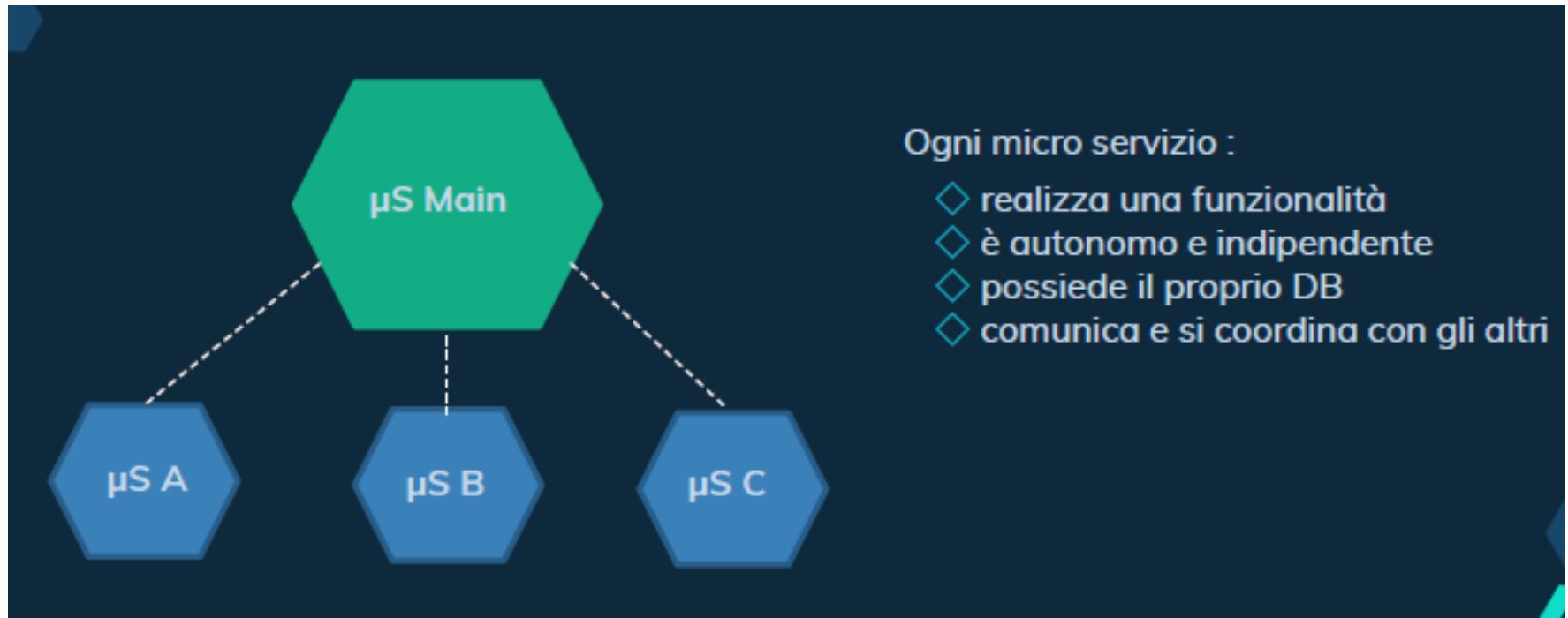


Se la app cresce

- ◇ Aumento della complessità generale
- ◇ Difficile trovare e risolvere bug
- ◇ Difficile effettuare cambiamenti
- ◇ Tempi estesi per il deployment
- ◇ Problemi di reliability

I Microservices nascono per trovare una soluzione

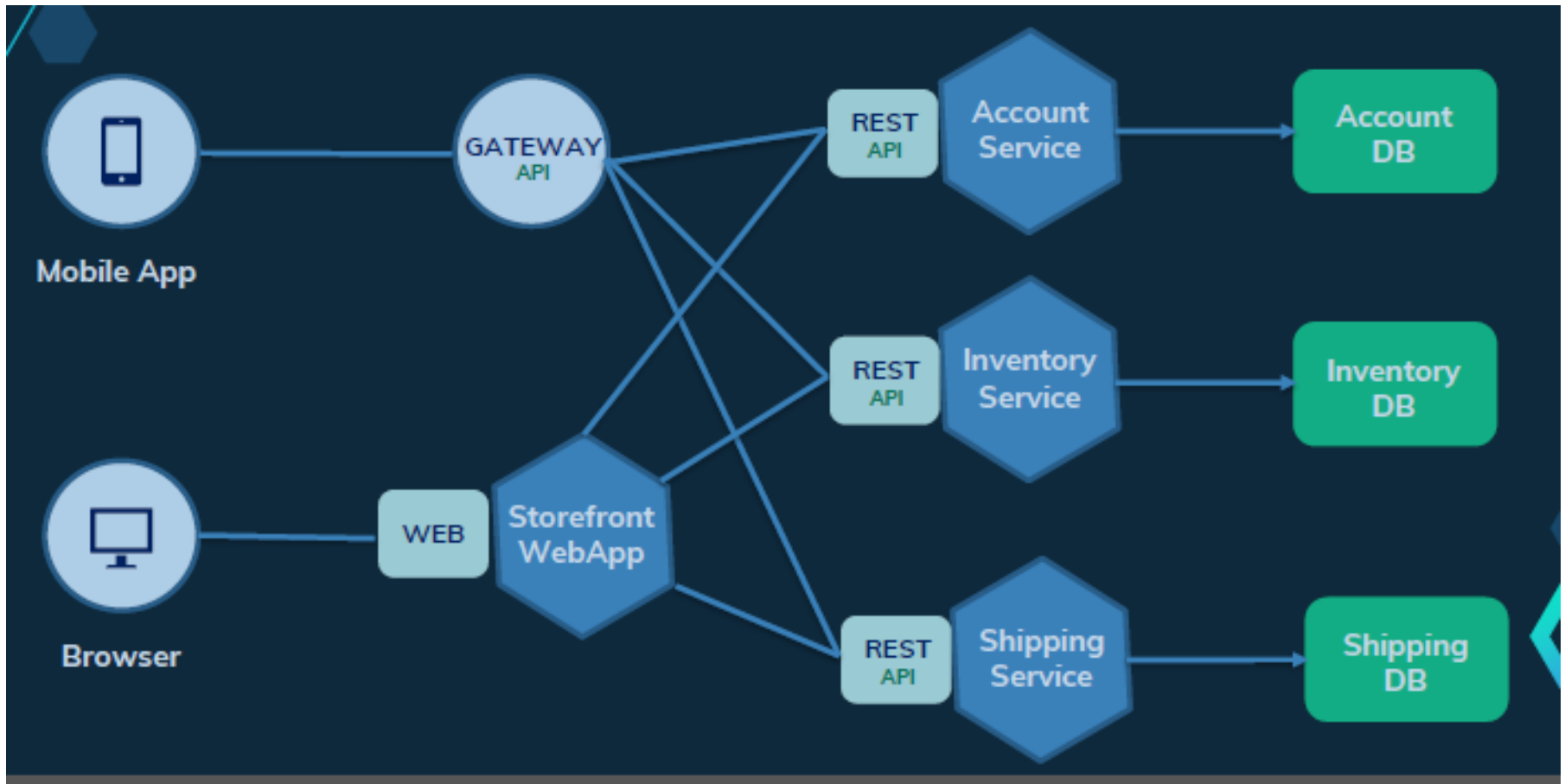
PROPRIETÀ DI UN MICROSERVICE



Ogni microservice può essere scritto in un linguaggio di programmazione diverso

REST = representational state transfer

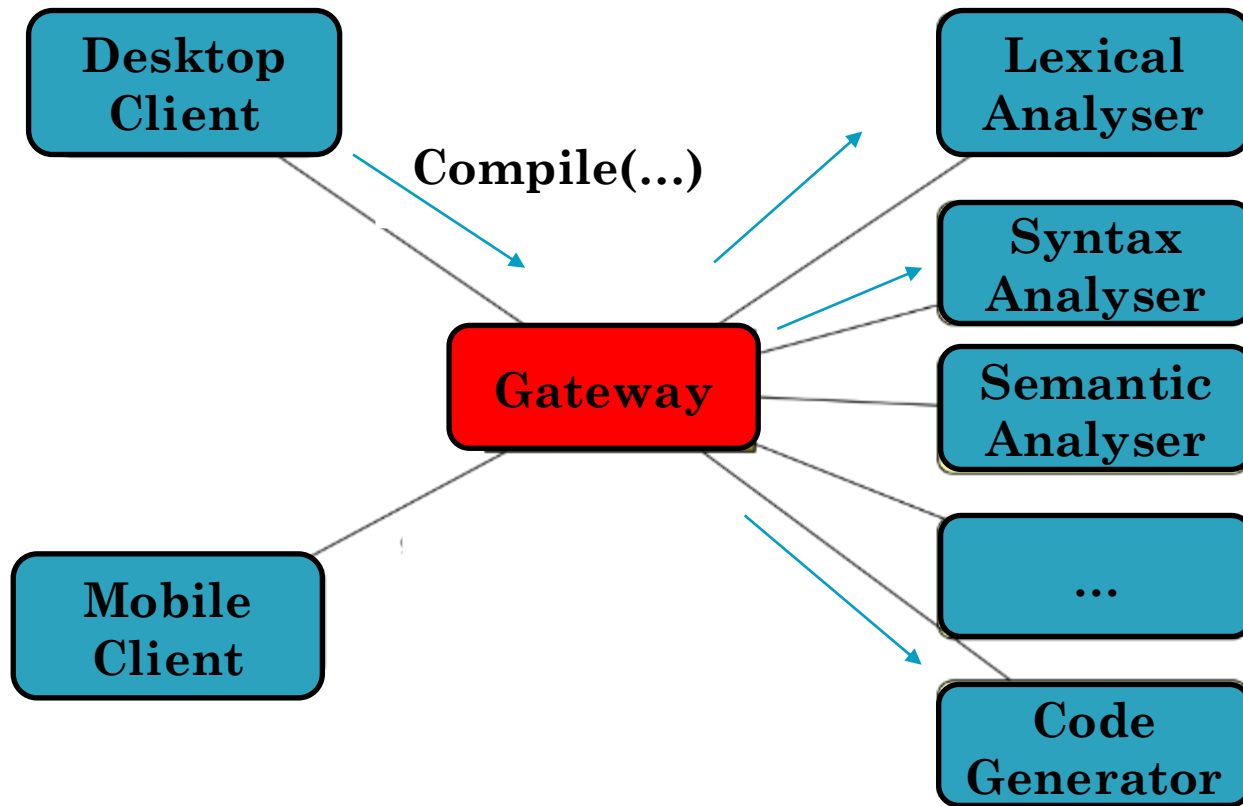
MICROSERVICES E-COMMERCE APP



Servizio finanziario, inventario e spedizione

GATEWAY

- L'API Gateway espone un'interfaccia verso i client
- Un client chiama ad esempio un unico servizio (via internet) e l'API gateway lo realizza chiamando gli 'n' servizi necessari) e invia l'output al client



REST

- Le applicazioni basate su REST utilizzano le richieste HTTP per inviare i dati (creazione e/o aggiornamento), effettuare query, modificare e cancellare i dati
- In definitiva, **REST utilizza HTTP** per tutte e quattro le operazioni CRUD (Create / Read / Update / Delete).



1 <http://www.acme.com/phonebook/UserDetails?firstName=John&lastName=Doe>

Esempio di chiamata di un microservice 'phonebook'
che restituisce i dettagli di un utente

QUALI PROBLEMI?

- ◇ Stabilire dimensione dei micro servizi
- ◇ Sviluppo del meccanismo di comunicazione tra i servizi (IPC)
- ◇ Esposizione ai disservizi di rete
- ◇ Gestione dello schema partizionato dei DB
- ◇ Difficoltà di testing
- ◇ Maggior consumo di risorse e memoria

SONO DAVVERO USATI?

The Amazon logo, featuring the word "amazon" in a black, lowercase, sans-serif font. A curved orange arrow is positioned below the letters "a" and "z", pointing from the "a" to the "z".

amazon

The eBay logo, featuring the word "ebay" in a lowercase, sans-serif font. The letters are colored: "e" is red, "b" is blue, "a" is yellow, and "y" is green. A small trademark symbol (TM) is located to the upper right of the "y".

ebay™

The Netflix logo, featuring the word "NETFLIX" in a bold, red, uppercase, sans-serif font.

NETFLIX



zalando

The Groupon logo, featuring the word "Groupon" in a bold, green, uppercase, sans-serif font. A small trademark symbol (TM) is located to the upper right of the "N".

GROUPON™

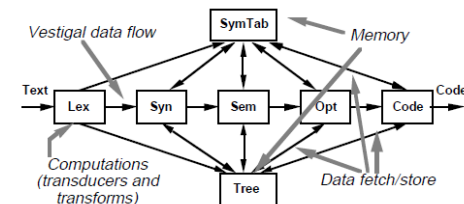
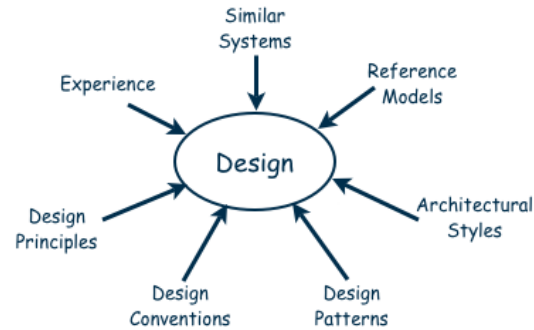


U B E R

RIASSUMENDO

○ Abbiamo visto:

- che il design ci permette di passare dallo spazio del **problema** alla spazio della **soluzione**
- che progettare un design **non è un task creativo!**
- quale è la differenza tra **High** e **Low** level design
- il concetto di **stile architettuale**
- diversi stili architettureali molto usati
 - ma non tutti ... impossibile ...
 - veloce introduzione dei Microservice
- che spesso i sistemi reali sono un mix di stili architettureali noti



LETTURE CONSIGLIATE

- An Introduction to Software Architecture, David Garlan and Mary Shaw, January 1994 (disponibile online)
- Cap. 5 del libro Software Engineering (fourth edition) S.L. Pfleeger and J. Atlee
- Cap. 11, 12, 13 del libro Ingegneria del software (8° edizione) I. Sommerville

